

TUGAS 8
Laporan Tugas Pembelajaran Mesin



Disusun oleh:

187241115 | Muhammad Naufal Zaki

S1 SISTEM INFORMASI
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS AIRLANGGA
2026

```

import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, classification_report,
    confusion_matrix, ConfusionMatrixDisplay
)

```

import streamlit as st: Modul untuk merender antarmuka pengguna berbasis web interaktif.

import pandas as pd dan import numpy as np: Pustaka untuk komputasi numerik dan manipulasi struktur data tabular berbasis DataFrame.

import matplotlib.pyplot as plt, import seaborn as sns, import plotly.express as px: Pustaka khusus visualisasi data untuk merender grafik analisis, baik statis maupun interaktif.

Dari modul sklearn, diimpor kelas DecisionTreeClassifier (algoritma pemodelan), train_test_split (pembagi dataset), serta modul dari sklearn.metrics seperti accuracy_score, confusion_matrix, dan classification_report untuk keperluan evaluasi Machine Learning.

```

st.set_page_config(
    page_title="Wine Quality Prediction",
    layout="wide",
    initial_sidebar_state="expanded"
)

# Custom CSS for Premium UI
st.markdown("""
<style>
    .main {
        background-color: #f8f9fa;
    }
    .stTabs [data-baseweb="tab-list"] {
        gap: 20px;
    }
    .stTabs [data-baseweb="tab"] {
        height: 50px;
        white-space: pre-wrap;
        background-color: #ffffff;
        border-radius: 5px 5px 0 0;
        gap: 1px;
    }
</style>
""")

```

```

        padding-top: 10px;
        padding-bottom: 10px;
        box-shadow: 0 2px 4px rgba(0,0,0,0.05);
    }
    .stTabs [aria-selected="true"] {
        background-color: #722F37 !important;
        color: white !important;
        font-weight: bold;
    }
    h1, h2, h3 {
        color: #722F37;
        font-family: 'Inter', sans-serif;
    }
    .metric-card {
        background-color: white;
        border-radius: 10px;
        padding: 15px;
        box-shadow: 0 4px 6px rgba(0,0,0,0.1);
        text-align: center;
        border-top: 4px solid #722F37;
    }
    .metric-value {
        font-size: 2rem;
        font-weight: 800;
        color: #722F37;
    }
    .metric-label {
        font-size: 1rem;
        color: #6c757d;
        font-weight: 500;
    }
</style>
""", unsafe_allow_html=True)

DATA_PATH = "WineQT.csv"

```

Konfigurasi antarmuka pengguna dilakukan melalui pemanggilan `st.set_page_config` yang mendefinisikan judul halaman “Wine Quality Prediction” serta pengaturan layout menjadi wide agar pemanfaatan ruang layar lebih optimal. Selain itu, pemanggilan `st.markdown` dengan parameter `unsafe_allow_html=True` dimanfaatkan untuk menyuntikkan CSS kustom yang bertujuan memperindah tampilan visual aplikasi, seperti pengaturan latar belakang kontainer, pembentukan lengkungan sudut elemen, serta desain metric card menggunakan efek box-shadow.

```

@st.cache_data
def load_and_train(path):
    df = pd.read_csv(path)

    # Drop 'Id' column if it exists
    if 'Id' in df.columns:
        df = df.drop(columns=['Id'])

    # Map quality to binary: 3,4,5 -> 0 (Bad), 6,7,8 -> 1 (Good)
    def map_quality(q):
        if q in [3, 4, 5]:
            return 0
        else:
            return 1

    df['quality_label'] = df['quality'].apply(map_quality)

    X = df.drop(columns=['quality', 'quality_label'])
    y = df['quality_label']

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=0
    )

    # Using Decision Tree exactly as requested
    model = DecisionTreeClassifier(random_state=0)
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    return df, X, X_train, X_test, y_train, y_test, y_pred, model

df, X, X_train, X_test, y_train, y_test, y_pred, model =
load_and_train(DATA_PATH)

```

Tahap prapemrosesan data diawali dengan penggunaan dekorator `@st.cache_data` pada fungsi `load_and_train`, yang berfungsi menyimpan hasil pemrosesan data ke dalam memori *cache* sehingga pembacaan data dan pelatihan model tidak dieksekusi berulang kali ketika terjadi *re-render* pada antarmuka. Dataset diekstraksi menggunakan `pd.read_csv` dari berkas `WineQT.csv` ke dalam *DataFrame* bernama `df`. Atribut `Id` kemudian dihapus menggunakan `df.drop(columns=['Id'])` karena tidak memiliki relevansi logis terhadap proses klasifikasi. Transformasi label dilakukan dengan memetakan nilai kualitas *wine* ke dalam skema klasifikasi biner, di mana skor 3, 4, dan 5 dikategorikan sebagai 0 (Bad), sedangkan skor 6, 7, dan 8 dikategorikan sebagai 1 (Good). Seluruh atribut independen dikelompokkan ke dalam variabel `X`, sementara atribut target disimpan pada variabel `y`.

```
def map_quality(q):
    if q in [3, 4, 5]:
        return 0
    else:
        return 1

df['quality_label'] = df['quality'].apply(map_quality)

X = df.drop(columns=['quality', 'quality_label'])
y = df['quality_label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)

# Using Decision Tree exactly as requested
model = DecisionTreeClassifier(random_state=0)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

return df, X, X_train, X_test, y_train, y_test, y_pred, model

df, X, X_train, X_test, y_train, y_test, y_pred, model =
load_and_train(DATA_PATH)
```

Proses partisi dataset dilakukan dengan memanggil fungsi `train_test_split` yang membagi data dengan proporsi 80% untuk data latih (`X_train, y_train`) dan 20% untuk data uji (`X_test, y_test`). Parameter `random_state=0` diatur untuk memastikan pemisahan data bersifat deterministik dan dapat direproduksi secara konsisten pada eksekusi berikutnya. Model klasifikasi kemudian diinisialisasi melalui objek `DecisionTreeClassifier(random_state=0)` dan dilatih menggunakan metode `fit(X_train, y_train)`. Setelah pelatihan selesai, metode `predict(X_test)` dijalankan untuk menghasilkan prediksi pada data uji yang disimpan dalam variabel `y_pred`.

```
col_logo, col_title = st.columns([1, 8])
with col_logo:
    st.markdown("<h1 style='text-align: center; font-size: 4rem;'>🍷</h1>", unsafe_allow_html=True)
with col_title:
```

```

st.title("Wine Quality Classification")
st.markdown(
    "Aplikasi cerdas berbasis Decision Tree Classifier untuk  
memprediksi "  
    "kualitas wine berdasarkan komposisi bahan kimianya."

```

Modul antarmuka dari fungsi `st.tabs` diinisiasi untuk menghasilkan partisi struktural pada web routing. Pendekatan ini menghasilkan tiga jendela pandang terpisah: (1) Eksplorasi Data, (2) Evaluasi Model, (3) Prediksi Kualitas Baru.

Komponen visual dasar juga dikonstruksi dalam metode ini untuk menampilkan representasi judul sistem pendukung keputusan.

```

with tab1:
    st.markdown("### 🍷 Dataset Overview")

    # Premium Metrics
    c1, c2, c3, c4 = st.columns(4)
    c1.markdown(f'<div class="metric-card"><div class="metric-label">Total Data</div><div class="metric-value">{len(df):,</div></div>', unsafe_allow_html=True)
    c2.markdown(f'<div class="metric-card"><div class="metric-label">Total Fitur</div><div class="metric-value">{X.shape[1]}</div></div>', unsafe_allow_html=True)
    c3.markdown(f'<div class="metric-card"><div class="metric-label">Wine Kategori "Bad" (3,4,5)</div><div class="metric-value">{(df["quality_label"]==0).sum()}</div></div>', unsafe_allow_html=True)
    c4.markdown(f'<div class="metric-card"><div class="metric-label">Wine Kategori "Good" (6,7,8)</div><div class="metric-value">{(df["quality_label"]==1).sum()}</div></div>', unsafe_allow_html=True)

    st.write("")
    st.markdown("#### Sampel Data Awal")
    st.dataframe(df.head(10), use_container_width=True)

    st.markdown("#### Visualisasi Distribusi Kualitas (Interaktif)")
    col_a, col_b = st.columns([1, 1])

    with col_a:
        # Plotly Bar Chart

```

```

        quality_counts =
df['quality_label'].value_counts().reset_index()
        quality_counts.columns = ['Quality', 'Count']
        quality_counts['Kategori'] = quality_counts['Quality'].map({0:
'Bad (3,4,5)', 1: 'Good (6,7,8)'})

        fig1 = px.bar(quality_counts, x='Kategori', y='Count',
color='Kategori',
                        color_discrete_map={'Bad (3,4,5)': '#EF553B',
'Good (6,7,8)': '#00CC96'},
                        title='Distribusi Kelas Quality')
        fig1.update_layout(showlegend=False)
        st.plotly_chart(fig1, use_container_width=True)

    with col_b:
        st.markdown("#### Matriks Korelasi (Fitur vs Kualitas)")
        # Plotly Heatmap
        corr = df.drop(columns=['quality']).corr()
        fig2 = px.imshow(corr, text_auto=".2f", aspect="auto",
color_continuous_scale='RdBu_r',
                        title='Correlation Heatmap')
        st.plotly_chart(fig2, use_container_width=True)

```

Pada tab Exploratory Data Analysis (EDA), sistem menampilkan metrik makro seperti jumlah total data, jumlah fitur, serta distribusi kelas Good dan Bad. Visualisasi distribusi kelas disajikan menggunakan px.bar yang bersifat interaktif. Selain itu, korelasi antar variabel independen dihitung menggunakan metode .corr() dan divisualisasikan dalam bentuk heatmap melalui px.imshow().

```

with tab2:
    st.markdown("### 📌 Evaluasi Model (Decision Tree)")

    accuracy = round(accuracy_score(y_test, y_pred) * 100, 2)

    c1, c2, c3 = st.columns(3)
    c1.markdown(f'<div class="metric-card"><div class="metric-
label">Akurasi Model</div><div class="metric-
value">{accuracy}%</div></div>', unsafe_allow_html=True)
    c2.markdown(f'<div class="metric-card"><div class="metric-
label">Data Training</div><div class="metric-value">{len(X_train):,}
(80%)</div></div>', unsafe_allow_html=True)
    c3.markdown(f'<div class="metric-card"><div class="metric-
label">Data Testing</div><div class="metric-value">{len(X_test):,}
(20%)</div></div>', unsafe_allow_html=True)

    st.write("")

```

```

col_cm, col_cr = st.columns([1, 1])

with col_cm:
    st.markdown("#### Confusion Matrix")
    cm = confusion_matrix(y_test, y_pred)
    fig3, ax3 = plt.subplots(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax3,
                xticklabels=['Bad (0)', 'Good (1)'],
                yticklabels=['Bad (0)', 'Good (1)'])
    ax3.set_xlabel("Prediksi Kualitas", fontsize=12)
    ax3.set_ylabel("Kualitas Aktual", fontsize=12)
    st.pyplot(fig3)

with col_cr:
    st.markdown("#### Classification Report")
    report = classification_report(y_test, y_pred,
output_dict=True)
    report_df = pd.DataFrame(report).transpose().round(2)
    st.dataframe(
        report_df.style.background_gradient(cmap='Blues',
subset=['f1-score']),
        use_container_width=True, height=350
    )

st.markdown("#### Visualisasi Decision Tree (3 Level Teratas)")
fig4, ax4 = plt.subplots(figsize=(25, 12))
plot_tree(
    model,
    feature_names=X.columns.tolist(),
    class_names=['Bad', 'Good'],
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=10,
    ax=ax4
)
ax4.set_title("Visualisasi Decision Tree", fontsize=16)
st.pyplot(fig4)

```

Tab evaluasi model menyajikan hasil pengukuran performa algoritma. Nilai akurasi dihitung menggunakan `accuracy_score`, sedangkan pola kesalahan prediksi divisualisasikan melalui `confusion_matrix` yang ditampilkan dengan bantuan `sns.heatmap`. Informasi metrik presisi, recall, dan F1-Score diekstraksi dari `classification_report` dan ditampilkan dalam bentuk `DataFrame` dengan pewarnaan gradien menggunakan `.style.background_gradient`. Struktur logika dari model Decision

Tree divisualisasikan menggunakan fungsi `plot_tree` dengan pembatasan kedalaman `max_depth=3` agar struktur pohon tetap terbaca.

```
with tab3:
    st.markdown("### 🍷 Form Prediksi Kualitas Wine")
    st.markdown("Geser slider di bawah ini sesuai dengan hasil uji lab wine untuk melihat prediksi kualitasnya.")

    with st.form("form_prediksi"):
        st.markdown("#### Parameter Kimiawi")
        col1, col2, col3 = st.columns(3)

        with col1:
            fixed_acidity = st.slider("Fixed Acidity", float(X['fixed acidity'].min()), float(X['fixed acidity'].max()), float(X['fixed acidity'].mean()))
            volatile_acidity = st.slider("Volatile Acidity", float(X['volatile acidity'].min()), float(X['volatile acidity'].max()), float(X['volatile acidity'].mean()))
            citric_acid = st.slider("Citric Acid", float(X['citric acid'].min()), float(X['citric acid'].max()), float(X['citric acid'].mean()))
            residual_sugar = st.slider("Residual Sugar", float(X['residual sugar'].min()), float(X['residual sugar'].max()), float(X['residual sugar'].mean()))

        with col2:
            chlorides = st.slider("Chlorides", float(X['chlorides'].min()), float(X['chlorides'].max()), float(X['chlorides'].mean()), format="%.3f")
            free_sulfur_dioxide = st.slider("Free Sulfur Dioxide", float(X['free sulfur dioxide'].min()), float(X['free sulfur dioxide'].max()), float(X['free sulfur dioxide'].mean()))
            total_sulfur_dioxide = st.slider("Total Sulfur Dioxide", float(X['total sulfur dioxide'].min()), float(X['total sulfur dioxide'].max()), float(X['total sulfur dioxide'].mean()))
            density = st.slider("Density", float(X['density'].min()), float(X['density'].max()), float(X['density'].mean()), step=0.001, format="%.4f")

        with col3:
            pH = st.slider("pH", float(X['pH'].min()), float(X['pH'].max()), float(X['pH'].mean()))
```

```

        sulphates = st.slider("Sulphates",
float(X['sulphates'].min()), float(X['sulphates'].max()),
float(X['sulphates'].mean()))
        alcohol = st.slider("Alcohol", float(X['alcohol'].min()),
float(X['alcohol'].max()), float(X['alcohol'].mean()))

    st.write("")
    submitted = st.form_submit_button("🍷 Prediksi Kualitas Wine",
use_container_width=True)

    if submitted:
        features = [[
            fixed_acidity, volatile_acidity, citric_acid,
residual_sugar,
            chlorides, free_sulfur_dioxide, total_sulfur_dioxide,
density,
            pH, sulphates, alcohol
        ]]

        input_df = pd.DataFrame(features, columns=X.columns)

        result = model.predict(input_df)[0]

        st.divider()
        st.markdown("### 🏆 Hasil Prediksi")

        col_res1, col_res2 = st.columns([1, 2])

        with col_res1:
            if result == 1:
                kategori_str = "GOOD"
                warna = "#00CC96"
            else:
                kategori_str = "BAD"
                warna = "#EF553B"

            st.markdown(
                f"""
                <div style="background-color: {warna}; padding: 20px;
border-radius: 10px; text-align: center; color: white;">
                    <h2 style="color: white; margin-bottom:
5px;">Quality Prediction</h2>
                    <h1 style="color: white; font-size: 4rem; margin:
0;">{kategori_str}</h1>

```

```

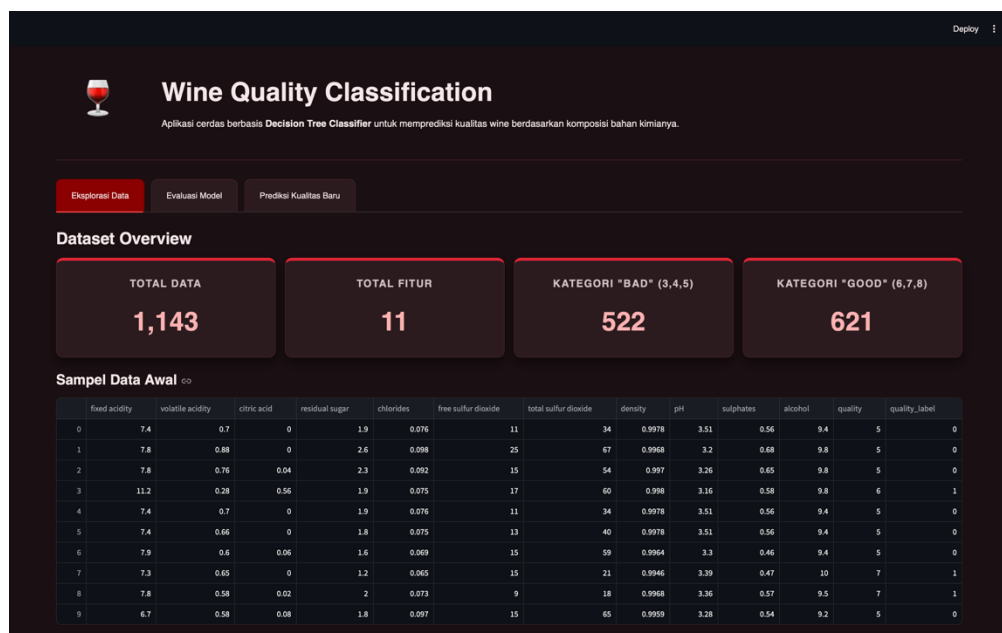
</div>
    """
    unsafe_allow_html=True
)

with col_res2:
    st.markdown("#### Detail Input:")
    st.dataframe(input_df, use_container_width=True)

    # Additional context about the score
    if result == 1:
        st.success("🌟 **Wine Berkualitas Baik (Good)** – Ciri khas komposisi sudah seimbang.")
    else:
        st.warning("⚠️ **Wine Berkualitas Kurang (Bad)** – Terdapat komposisi kimia yang belum optimal.")

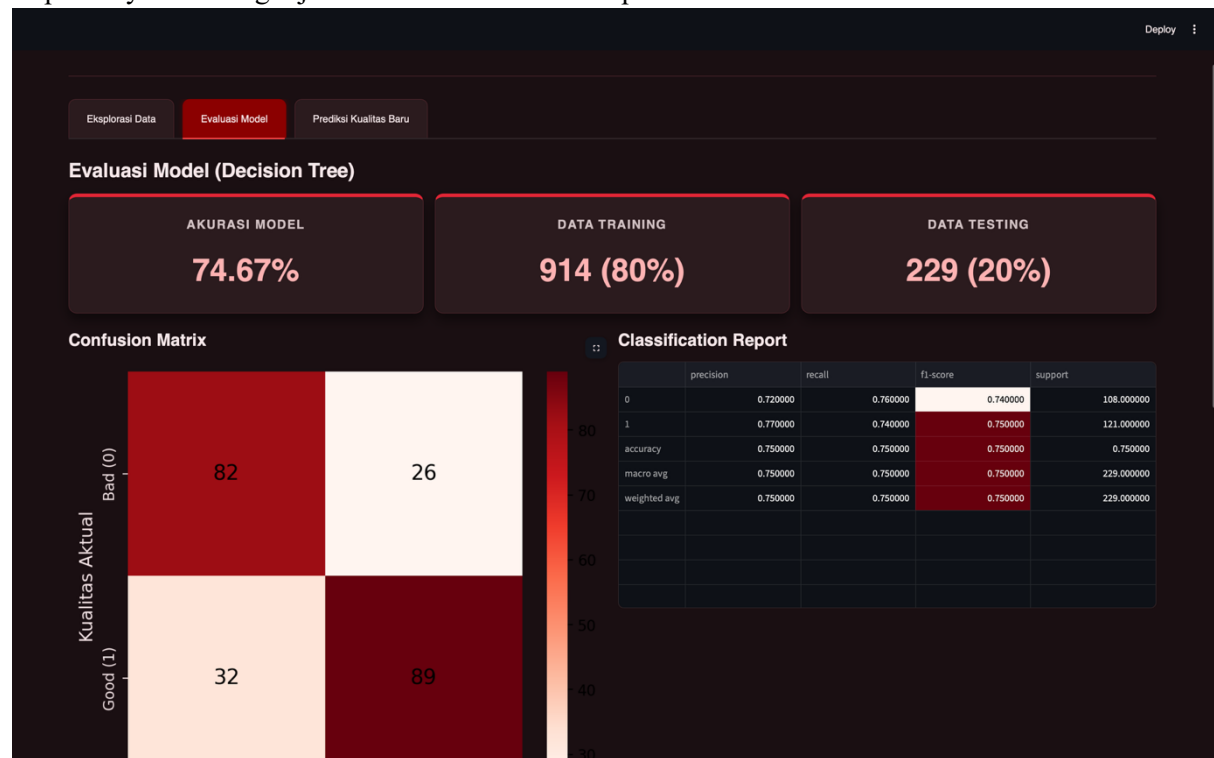
```

Pada tab simulasi prediksi, mekanisme state management diimplementasikan melalui `st.form("form_prediksi")` yang menunda proses komputasi hingga tombol konfirmasi ditekan. Input pengguna dikumpulkan melalui serangkaian `st.slider` yang batas nilai minimum, maksimum, dan nilai awalnya diambil langsung dari distribusi data asli pada `X`. Ketika tombol `st.form_submit_button` ditekan, seluruh nilai input dikonversi menjadi sebuah `DataFrame` bernama `input_df`. Model kemudian melakukan prediksi melalui `model.predict(input_df)`, dan hasilnya ditampilkan secara kondisional, di mana nilai 1 ditampilkan sebagai “GOOD” dengan warna hijau, sedangkan nilai 0 ditampilkan sebagai “BAD” dengan warna merah.



Dataset yang digunakan terdiri dari 1.143 data observasi dengan 11 fitur kimia yang merepresentasikan karakteristik fisikokimia setiap sampel wine. Seluruh fitur bersifat numerik dan terukur secara laboratoris, sehingga sangat sesuai untuk pemodelan berbasis Machine Learning.

Label kualitas ditransformasikan dari skala ordinal (3–8) menjadi klasifikasi biner, yaitu “Bad” untuk kualitas 3, 4, dan 5 sebanyak 522 data, serta “Good” untuk kualitas 6, 7, dan 8 sebanyak 621 data. Pendekatan ini mempermudah model dalam membedakan dua kategori utama kualitas wine. Sebelas fitur yang digunakan meliputi fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, dan alcohol. Dataset ini merupakan dataset klasik Red Wine Quality yang dipublikasikan oleh UCI Machine Learning Repository dan sering dijadikan referensi dalam eksperimen klasifikasi.



Proses split data dilakukan dengan komposisi 80% untuk training (914 data) dan 20% untuk testing (229 data). Artinya, model belajar dari sebagian besar data terlebih dahulu, lalu kemampuannya diuji menggunakan data baru yang belum pernah dilihat.

Hasil evaluasi menunjukkan akurasi sebesar 74,67%, dengan nilai precision, recall, dan F1-Score rata-rata di angka 0,75. Ini menandakan performa model cukup seimbang dalam ketepatan prediksi, kemampuan mengenali tiap kelas, dan konsistensi di antara keduanya.

Jika dilihat dari confusion matrix, model berhasil menebak dengan benar 82 data Bad sebagai Bad (True Negative) dan 89 data Good sebagai Good (True Positive). Namun, masih terdapat kekeliruan pada 26 data Bad yang diprediksi Good (False Positive) serta 32 data Good yang diprediksi Bad (False Negative).

Akurasi di angka ini tergolong wajar untuk model Decision Tree yang digunakan tanpa tuning. Performa tersebut masih bisa ditingkatkan, misalnya dengan melakukan pruning melalui pengaturan max_depth dan min_samples_split, atau dengan beralih ke algoritma yang lebih kuat seperti Random

Forest.

Deploy



Wine Quality Classification

Aplikasi cerdas berbasis **Decision Tree Classifier** untuk memprediksi kualitas wine berdasarkan komposisi bahan kimianya.

Explorasi Data

Evaluasi Model

Prediksi Kualitas Baru

Form Prediksi Kualitas Wine

Geser slider di bawah ini sesuai dengan hasil uji lab wine untuk melihat prediksi kualitasnya.

Parameter Kimiawi

Fixed Acidity	11.00	Chlorides	0.087	pH	3.31
Volatile Acidity	0.53	Free Sulfur Dioxide	15.62	Sulphates	1.30
Citric Acid	0.27	Total Sulfur Dioxide	45.91	Alcohol	8.40
Residual Sugar	2.53	Density	0.9967		

Prediksi Kualitas Wine

Pengguna dapat memasukkan nilai parameter kimia wine melalui slider interaktif yang tersedia pada antarmuka. Setelah seluruh nilai diatur, pengguna cukup menekan tombol “Prediksi Kualitas Wine”, dan sistem akan langsung menampilkan hasil klasifikasi Good atau Bad berdasarkan inferensi dari model.